

BRISKSNAPSHOT: A Live Demo of Join-Backed Semantic Windows for Streaming AI Pipelines

Anonymous Author(s)

ABSTRACT

Streaming AI applications need to keep vector state fresh while exposing that state in a form that downstream services can inspect and act on. BRISKSNAPSHOT is a live demonstration of a unified semantic-window system built from the SAGE orchestration codebase and the SageFlow vector-stream runtime [4, 5, 10]. The demo follows public vulnerability-intelligence reports as they are normalized, embedded with BGE, joined inside a bounded semantic window, and surfaced as snapshot, routing, and escalation contracts for an LLM-facing application. Unlike a static retrieval demo, BRISKSNAPSHOT lets the audience see how new events change the active window, which vector joins support a cluster, and what compact evidence is handed to the generator. The live walkthrough stays small enough for narration, while a 3,000-record NVD replay shows measured runtime behavior under real embeddings.

CCS CONCEPTS

• **Computer systems organization** → **Parallel architectures**; • **Information systems** → **Stream management**; • **Computing methodologies** → **Artificial intelligence**.

KEYWORDS

stream processing, vector search, AI middleware, semantic windows, demonstration

1 INTRODUCTION

Large-model applications increasingly run as long-lived dataflows rather than as isolated prompt calls. A vulnerability assistant receives new advisories while an investigation is still open; a retrieval pipeline refreshes its vector neighborhood between two user questions; an on-call agent must decide whether a new report belongs to a known incident or deserves escalation. In these settings, the model-facing component should not receive an opaque vector index or a free-form prompt dump. It needs bounded, inspectable evidence: which records are retained, which records matched, which sources support the cluster, and which action the system recommends.

Stream processors provide scalable dataflow execution [1, 2], while vector indexes provide efficient approximate search [6, 7, 9]. Retrieval-augmented systems further show how language models benefit from external evidence [3, 8]. What is still awkward in many prototypes is the boundary between streaming state and AI orchestration. Semantic state is often rebuilt outside the component that observes the stream, so a demo can show an answer but not the live runtime evidence that made the answer possible.

BRISKSNAPSHOT demonstrates a narrower and more observable boundary. The system owns the active semantic window, runs vector joins as events arrive, and exports only compact contracts to the application layer. A contract is not another prompt; it is a structured object with cluster identity, supporting neighbors, source coverage,

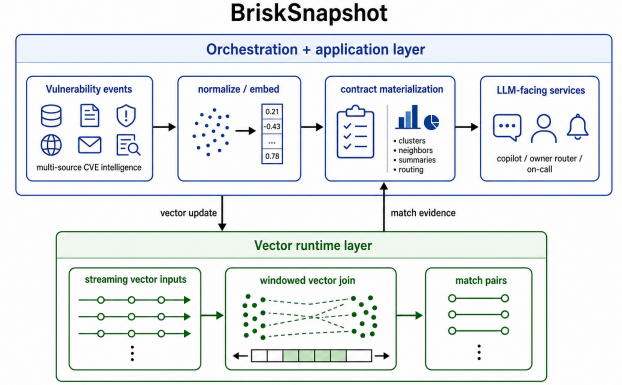


Figure 1: BRISKSNAPSHOT architecture. Events are normalized and embedded, a persistent vector runtime owns the semantic window and join pairs, and the application layer receives bounded contracts.

novelty, route, priority, and action fields. Those fields are then rendered in the UI and placed into the LLM context, so the audience can trace an answer back to runtime-owned evidence rather than to hidden prompt construction. The demo is built around the question an audience naturally asks during a live system demonstration: when the next event arrives, what changes, why did it change, and what can the upper pipeline safely do with the change?

The paper keeps the original demo narrative but makes three pieces explicit. First, it describes the system architecture as a single integrated prototype rather than two adjacent components. Second, it expands the live demonstration path so that the UI, presenter script, and pipeline stages have clear roles. Third, it adds prepared evidence from real public records and real embedding vectors, including a parallelism sweep that shows both the benefit and the current cost of clustered multicast joins without turning the paper into a full benchmark paper.

2 SYSTEM OVERVIEW

Figure 1 shows the implemented path. BRISKSNAPSHOT takes heterogeneous vulnerability reports from multiple sources, maps each report into a 384-dimensional BAAI/bge-small-en-v1.5 embedding, appends the record into a bounded semantic window, and drains join pairs produced by the persistent runtime. The output is then folded into application-level contracts and, in the API-backed mode, into a bounded prompt context for GLM generation. This organization is intentionally close to the way the live demo is presented: the audience first sees an event enter the pipeline, then sees the runtime evidence that connects it to prior records, and finally sees the contract that an application can consume.

Table 1: Runtime boundary exposed by the demo.

Component	Demo-visible role
Ingestion layer	Source replay, BGE embedding cache, event metadata
Window manager	Bounded retention, reset/cleanup, snapshot export
Join runtime	Left-right append, vector comparison, pair emission
Contract layer	Snapshot, owner route, escalation signal
UI and evidence	Live controls, replay trace, scale summaries

2.1 Design Goals

The first design goal is *observable freshness*. A static retrieval index can demonstrate relevance, but it hides whether the context is current. BRISKSNAPSHOT keeps the current window visible: retained records, emitted pairs, active clusters, and newest evidence are all displayed as runtime state rather than as post-hoc explanation text.

The second goal is *bounded handoff*. Model-facing services should not receive an unbounded list of nearest neighbors. The system exports a small set of fields that can be logged, routed, or summarized: cluster identity, source coverage, supporting neighbor identifiers, novelty, route, priority, and action. This turns the semantic window into an application interface rather than a private runtime detail.

The third goal is *demo reproducibility*. Live demos are convincing when they are interactive, but papers also need evidence that the observed behavior is not a hand-picked screen. The same replay driver used for the UI can run public NVD/CISA inputs, cached real embeddings, threshold variants, and window-size variants. The prepared figure in Section 4 is generated from those replay artifacts.

2.2 Runtime Boundary

Table 1 summarizes the system boundary. The ingestion layer gives the runtime comparable vectors and stable identifiers. The window manager owns which events are still active and which events have left the current view. The join runtime is the only component that creates semantic match pairs. The contract layer interprets those pairs for an application: a dense multi-source cluster becomes an incident snapshot, a cluster with a clear owner becomes a routing contract, and a high-severity novel event becomes an escalation contract.

Cluster construction uses connected components over returned join pairs. Clusters are sorted by size and average severity before export, which makes the display stable enough for a presenter to narrate. The default incident rule requires at least three events, average severity above 0.70, and at least two source classes. The emerging-risk rule fires when a high-severity event remains novel enough to deserve attention even before a larger cluster forms.

3 LIVE DEMONSTRATION

The live scenario uses a narrated subset of public vulnerability-intelligence records from NVD, vendor advisories, CISA warnings, emergency alerts, and research notes. The reports cover familiar incident families such as PAN-OS, XZ Utils, OpenSSH, PHP-CGI,

Table 2: Presenter script for the live demo.

Step	What the audience sees
Replay	Source, severity, timestamp, and embedding handoff for the current event
Runtime	Retained records, emitted pairs, and nearest neighbors inside the active window
Snapshot	Cluster membership, source coverage, novelty, and representative evidence
Contract	Owner route, priority, recommended action, and supporting event identifiers
Variant	How threshold, window size, or input order changes the visible contract stream

Fortinet VPN, and TeamCity. This domain is useful for a systems demo because records arrive from different sources, use different language, and still need to be grouped quickly enough for operator-facing workflows. The larger evidence run uses 3,000 NVD records published in the first quarter of 2024, embedded once with BGE and replayed through the same SageFlow runtime path.

3.1 Audience Path

Figure 2 shows the UI used in the demonstration. The left side controls the replay and displays the active pipeline trace. The right side shows contract scenarios and runtime evidence. The important UI choice is that the pipeline is not decorative: each row corresponds to a stage that changes system state. The event stage introduces a source report; the embedding stage makes the report comparable; the append stage changes the retained window; the join stage exposes which prior records matched; the contract stage shows what the application receives.

The presenter starts from an empty runtime and runs the stream one event at a time. Early events trigger emerging-risk contracts because the system has not yet seen enough related evidence. As reports from additional sources arrive, the same incident families become connected clusters and the UI switches from novelty to correlation. The audience can pause the replay, inspect the nearest neighbors supporting a cluster, and compare the current snapshot with the contract text that would be passed to a downstream service.

Table 2 is the actual demo script. The walkthrough is designed to answer five questions in order. What just arrived? What did the runtime retain? Which records joined? What compact evidence was exported? What changes when the audience asks for a different parameter setting? This is the same structure used in many systems demonstrations: start with a concrete user path, reveal internal state at the moment it matters, and close by showing robustness beyond the exact path shown on screen.

3.2 Demonstration Scenarios

The snapshot scenario shows why the bounded window matters. When the PAN-OS reports arrive from a vendor advisory, NVD, an emergency warning, and a research note, the system connects them into a single family and exports a compact summary with source coverage. The UI lets the audience see that the summary is not a generic LLM explanation; it is grounded in event identifiers and emitted join pairs.

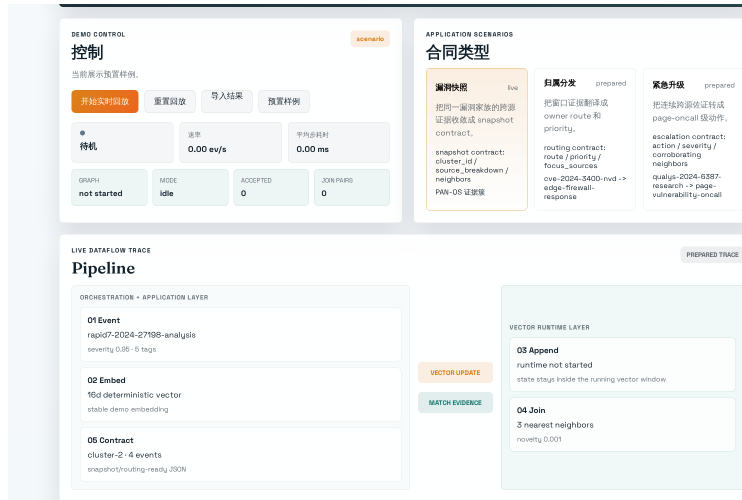


Figure 2: Web interface for the demo. Controls, contract modes, runtime evidence, and prepared experiment summaries remain visible so the audience can connect UI actions with the semantic-window state underneath.

The routing scenario shows how the same evidence can be converted into an owner decision. A VPN-related cluster is routed to the network perimeter owner because the source mix and severity cross the routing threshold. The presenter can open the route card, inspect the supporting neighbors, and show that route changes are tied to runtime evidence rather than keyword matching.

The escalation scenario focuses on high-severity novelty. A new report may not yet have enough neighbors for a dense cluster, but it can still become actionable if severity is high and novelty remains above threshold. This scenario is important for audience trust: the system is not only finding repeated topics; it also preserves a path for emerging risks that have not yet accumulated multiple sources.

4 PREPARED EVIDENCE

The narrated walkthrough is intentionally small enough for a live audience. To show more than a toy sample, the prepared evidence uses 3,000 public NVD records from 2024-Q1. Embeddings are produced by BAAI/bge-small-en-v1.5 through SentenceTransformers, normalized to 384 dimensions, sharded for artifact storage, and replayed from cache so the runtime experiment is deterministic. A sampled vector was recomputed from the original text and matched the cache within floating-point roundoff, which guards against reporting placeholder vectors.

4.1 Workloads and Metrics

The runtime workload contains 3,000 NVD records and 6,000 vector insertions because each event is replayed into the two input sides of the join. We report feed-and-drain latency, event throughput, speedup relative to one worker at the same window size, and emitted pair count. The experiment uses `clustered_join`, a low similarity threshold of 0.5, multicast fanout two, and sequence timestamps so window sizes correspond to recent vector arrivals rather than calendar time.

Figure 3 shows the main result. With a 512-arrival window, throughput improves from 22.7k events/s at $p=1$ to 26.4k events/s

Window	Parallelism	Throughput	Speedup	Pairs
512	1	22.7k ev/s	1.00×	6,480
512	2	26.4k ev/s	1.16×	8,450
512	4	20.1k ev/s	0.89×	12,841
512	8	11.9k ev/s	0.52×	28,509
1024	1	18.0k ev/s	1.00×	9,025
1024	2	28.1k ev/s	1.57×	7,260
1024	4	20.8k ev/s	1.16×	12,667
1024	8	12.3k ev/s	0.69×	30,137

Figure 3: Prepared runtime result on 3,000 real NVD records with BGE embeddings. The clustered join uses threshold 0.5 and multicast fanout two; $p=2$ gives the best throughput in the current prototype, while larger p increases multicast work and emitted pairs.

at $p=2$. With a 1,024-arrival window, throughput improves from 18.0k events/s to 28.1k events/s, a 1.57× speedup. Higher parallelism exposes a current engineering limit: multicast increases the number of emitted candidates and synchronization work, so $p=4$ and $p=8$ do not monotonically improve throughput. This is still useful demo evidence because the UI and paper can explain both the concurrency benefit and the cost of broader candidate coverage.

Table 3 lists the parameter variants that the presenter can switch to after the main walkthrough. These variants are included because a demo paper should not only describe the happy path. A tighter threshold tests whether the interface still explains why fewer clusters appear. A shorter window tests whether bounded retention is visible. Reordered input tests whether the system preserves incident families while changing the moment at which contracts become visible.

Table 3: Prepared parameter variants used during the demo.

Variant	Expected visible effect
Tighter threshold	Fewer join pairs, fragmented clusters, fewer correlated contracts
Shorter window	Older incident families leave the snapshot earlier
Reordered input	Same topic families, different contract reveal times
Clustered fanout	More candidate coverage, higher emitted-pair count, more runtime overhead

4.2 What the Evidence Shows

The prepared evidence supports three claims. First, the live path has enough data to show real behavior: reports from multiple source classes produce incident families, cross-source clusters, and both correlation and novelty contracts. Second, the runtime boundary remains measurable on a larger public stream: the same cached embeddings and runtime API process thousands of records and expose throughput, latency, and emitted pairs. Third, the UI exposes the relevant system effect when parameters change. A demo attendee does not need to infer behavior from a hidden log; the state transition is visible through retained records, match pairs, clusters, exported contracts, and the final generated answer.

5 DISCUSSION AND CONCLUSION

BRISKSNAPSHOT is a demonstration of a system boundary: semantic-window state remains inside a persistent vector runtime, while the application receives bounded contracts that are stable enough to inspect, log, route, and summarize. The key point is not that vector search can find similar records; the key point is that streaming vector evidence can become observable application behavior. The demo therefore emphasizes the handoff from event stream to runtime evidence to contract output.

There are two takeaways for attendees. The first is operational: a semantic window should expose enough state for a human to understand why an application-level decision changed. In BRISKSNAPSHOT, the presenter can point from a contract back to retained events and join pairs, which makes the system easier to debug than a pipeline that only displays generated text. The second takeaway is architectural: keeping the join state inside a long-lived runtime avoids rebuilding semantic context at every application step. This is what lets the demo show freshness, boundedness, and contract emission as one continuous behavior.

The demo also makes its current limitations visible. The embedding cache is a replay artifact built from a real model; a deployed system would refresh vectors online and monitor embedding drift. The current clustered join profile benefits from modest parallelism but pays overhead at larger multicast fanout, which we expose rather than hide. The contract rules are intentionally simple threshold rules, which makes the UI explainable but leaves room for learned policies or organization-specific routing constraints. These limitations are useful in a demonstration setting because they separate the systems contribution from policy choices that can vary across applications.

The current prototype focuses on vulnerability intelligence because the domain makes semantic grouping and escalation easy to understand. The same pattern applies to other streaming AI tasks where new evidence must be incorporated during a long-running workflow: customer-support incidents, scientific-monitoring feeds, or operational telemetry. Future extensions will make the contract rules learned or policy-driven, but the live demo already exposes the essential systems question: how can an AI pipeline keep semantic context fresh without hiding the runtime state that justifies its actions?

REFERENCES

- [1] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, Sam Whittle, Robert Wilfong, and Lukasz Zajac. 2015. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1792–1803.
- [2] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache Flink: Stream and Batch Processing in a Single Engine. *IEEE Data Engineering Bulletin* 38, 4 (2015), 28–38.
- [3] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. Retrieval Augmented Language Model Pre-Training. In *Proceedings of the 37th International Conference on Machine Learning*. PMLR, Virtual Event, 3929–3938.
- [4] IntelliStream Research Group. 2026. SAGE: Streaming-Augmented Generative Execution. GitHub repository. <https://github.com/intellistream/SAGE>.
- [5] IntelliStream Research Group. 2026. SageFlow: Vector-Native Stream Processing Engine. GitHub repository. <https://github.com/intellistream/sageFlow>.
- [6] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.
- [7] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 9459–9474.
- [9] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.
- [10] Shuhao Zhang et al. 2026. SAGE: Streaming-Augmented Generative Execution. In *Proceedings of the 43rd International Conference on Machine Learning*. PMLR, To appear.